

Lecture 5

**Consistency and Concurrency
Control, and Fault Tolerance**

COMP2211: Distributed Systems

Thiru Vasantam

Topic 6 continued: Concurrency and Consistency Control

COMP2211

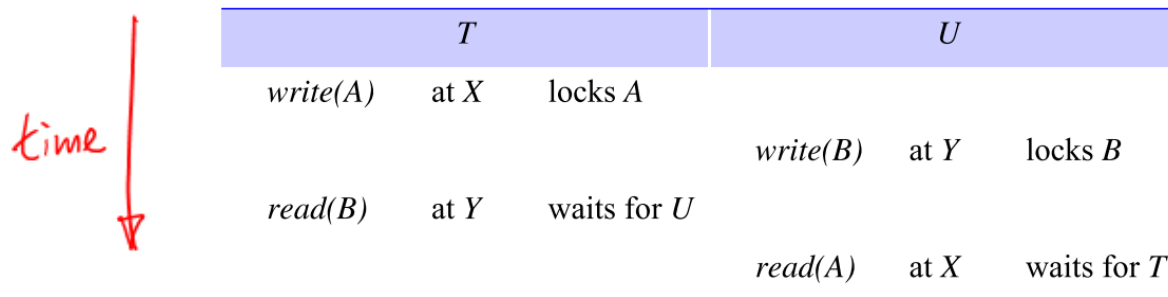
Distributed Systems

Topic 6: Concurrent Events and Consistency Control

Thiru Vasantam

Locking

- Grant a lock
 - Locks on an object are held locally (in a server locally)
 - Local lock manager can decide whether to grant a lock or make the requesting transaction wait
- Issues with Distributed Transaction (DT):
 - A DT acquires resources located at different servers
 - Cannot release any locks until the transaction has been committed or aborted **at all servers** involved in the transaction
 - Objects remain locked and are unavailable for other transactions during the atomic commit protocol (means allocate resources at all related nodes for a transaction)



Deadlock Situation

Discussion: Concurrent Operations

- In a single machine:
 - Concurrent operations (events), no matter originated from different or the same machine, are handled by the time-sharing feature of an operating system
 - Operations are implicitly executed one by one in a series under a single clock, i.e., order of operations is well-defined
- In a distributed system:
 - Concurrent operations may run on different machines, i.e., distributed transactions
 - Order of operations cannot be easily sorted out due to overlapping operations, clock synchronization problem (no global clock), network latency and message loss, etc.
 - Clock Sync.: NTP (Network time protocol) [ref: Chap. 14]

Timestamp Ordering

- **Timestamp**

- Assign a timestamp to each transaction when it starts.
- **Serial equivalence:** is enforced by committing the versions of objects (i.e., replicated objects) in the order of the timestamps of transactions that accessed them.
- If 't1' is time-stamp for T and 't2' is time-stamp for U, then T accesses an object before U when $t1 < t2$
- Requirement: Globally unique timestamps

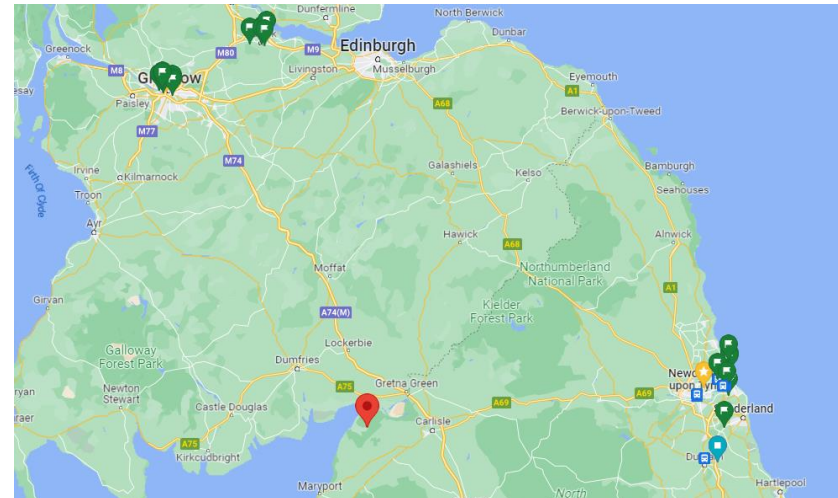
Timestamp Ordering

- **Distributed Transaction**

- Servers of distributed transactions are jointly responsible for ensuring that they are performed in a serially equivalent manner.
- If transaction T accesses an object before U at a server, then the same ordering is maintained at other servers for accessing conflicting objects, i.e., T accesses an object before U at any server.
- To achieve the same ordering at all the servers, the coordinators must agree on the ordering of their timestamps.
- Issue: In practice, a timestamp is usually assigned by a local server, generating a **<local timestamp, server-id>** pair.

NTP as a Distributed System

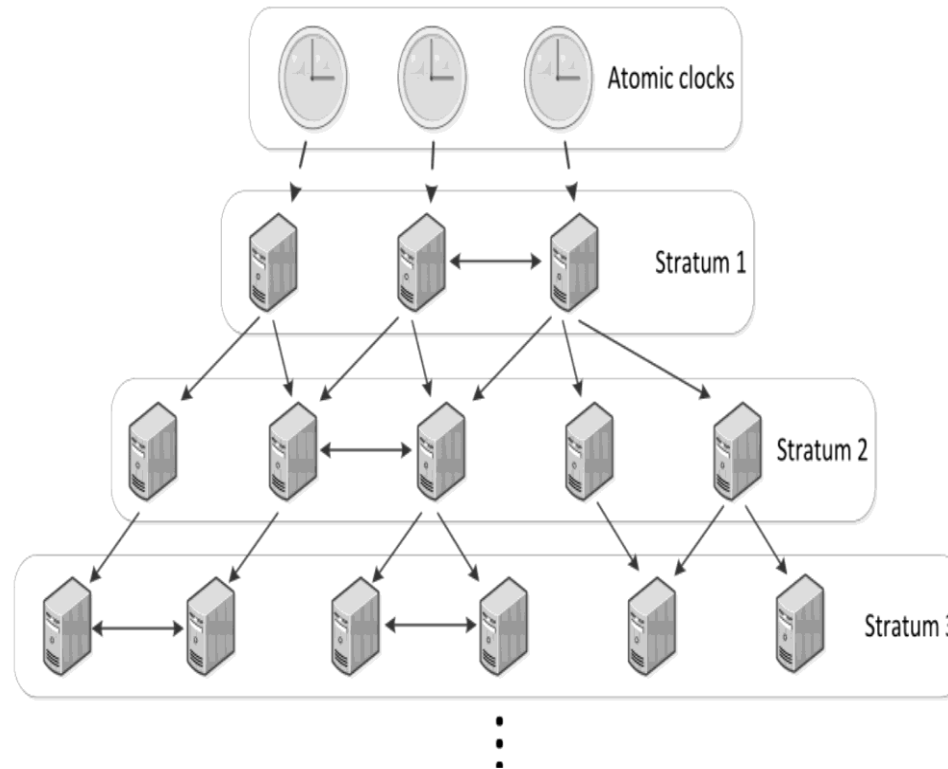
- Network Time Protocol (NTP) is a protocol used to synchronize the clocks of computers over a network. NTP uses a hierarchical structure of time servers – stratum levels, to ensure that accurate time info. is propagated throughout the network.
- **Accuracy:** Sub-millisecond - local area network (LAN); within a few milliseconds - over Internet
- e.g., **Atomic clocks** (highly accurate clocks): MSF time and frequency standard (Anthorn, Cumbria, UK)



NTP as a Distributed System

- **Stratum 1, 2:** Usually operated by research institutions or councils, and major ISPs
- **Stratum 3:** Usually operated by smaller data centres or ISPs
- Servers in Stratum 'i' get messages from Stratum 'i-1' to correct their clocks, based on the time in clocks of Stratum 'i-1' servers and the transmission delay of sending messages between the two layers

HOW NTP WORKS?



Consistency

Correctness of Replicated Objects

- If a system only maintains a single copy for each object, object correctness simply relates to the sequence of operations applied on the object.
- Correctness of Replicated Objects is challenging:
 - Object copies maintained by different machines, where each machine may receive operations with a different order (due to network latency, clock synchronization errors, or message loss / re-transmission).

An Example on Synchronization Errors

- Suppose that you want to perform some operations, operation x on server A at 10.02 AM and operation y on server B at at 10.04AM (these times are according to atomic clocks)

- There are clock synchronization errors at servers (they maybe in different parts of the world)

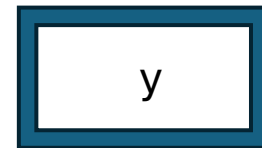
10.02AM on server A is 10.06AM on atomic clock

10.04AM on server B is 10.03AM on atomic clock

- The actual implementation of operations is in the reverse order; first y and then x



A



B

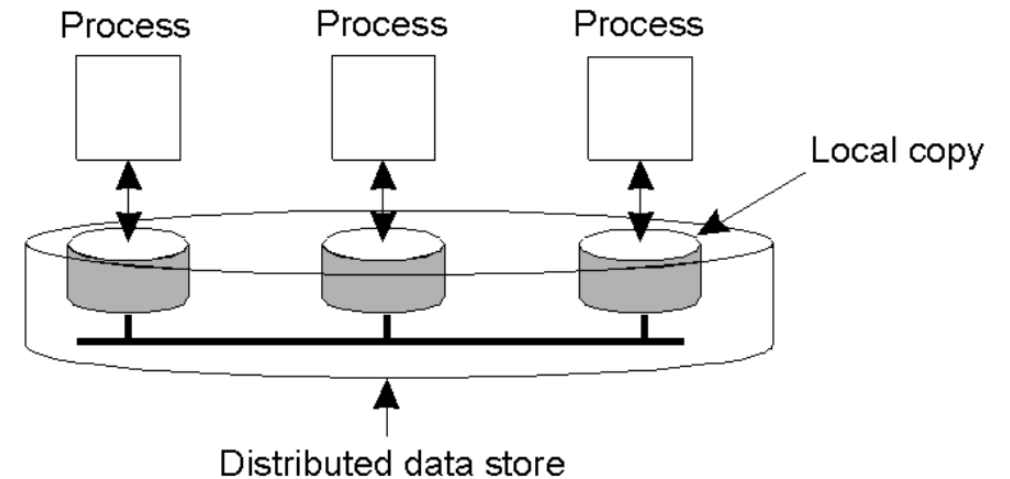
Consistency of Data

Consistency model

A contract between a (distributed) data store and processes, in which the data store specifies precisely what the results of read and write operations are in the presence of concurrency.

- All models attempt to return the results of the **last write for a read operation**
- Differ in how “last” write is determined/defined at each disk. To loosen consistent requirements, we have several notions of consistencies.

- Processes access local copy for quick responses
- The last write operation could be different for the local copies in the figure



Consistency Control

Issues with distributed systems (DSs):

- DSs rely on replicating shared objects, or allowing **concurrent access** at many nodes
- If concurrent accesses of (shared) objects are not carefully controlled, object accesses may be executed in an order different from user expectation, generating incorrect results

Modeling of Consistency Control:

- Consistency Model: Define rules for the **apparent order and visibility** of updates, and it is a continuum with trade-offs

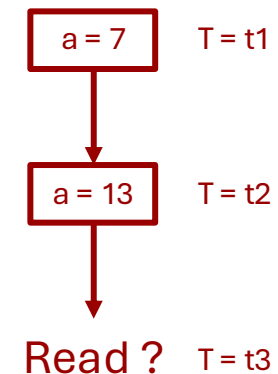
Strict Consistency

The strongest consistency model

- Any read on a data item X returns a value corresponding to the result of the most recent write on X
- A write is immediately visible to all processes, assumes no transmission delays
- Difficult to achieve in real systems (network delays can be variable)

Requirement:

- Need an absolute global time
 - Such that “most recent” becomes unambiguous
 - This behavior can be observed in a single processor
 - `a=7; a=13; print(a); { *has* to print 13 as output}` even if $t_1 - t_2$ is small
- Practically, can only be implemented within one single machine



In replication systems: It is challenging to achieve in systems with distributed replicas due to: global clock synchronisation issues, lack of a single ordering of operations (maybe a centralized control decides it), differences in the recent write operations at replicas.

Linearizability

- Two processes P1 and P2 execute different operations sequentially

P1: x1 $\xrightarrow{\text{time}}$ x3
 x2

P2: y1 y2 y3 y4

- Linearizability:** Find virtual interleaving of operations as if they are executed on a single machine and the data is consistent with real times:

x1 y1 y2 x2 y3 x3 y4

- Such that if $TS(x) < TS(y)$ then $OP(x)$ should precede $OP(y)$ in the sequence, where TS is time-stamp and OP is operation (read or write).

Linearizability – Definition

- There exists a **virtual canonical execution** – the interleaved operations against a virtual single image of shared (and particularly replicated) objects, and that:
 - Each client sees a view of the shared objects that is consistent with that single image
 - i.e., results of the client's operations make sense as they occur within the interleaving
- In distributed systems, operations are **Linearizable** if:
 - The interleaved sequence of operations **meets the specification** of a (single) correct copy of the objects (means read and write operations should make sense).
 - The order of operations in the interleaving is **consistent with the real times** at which the operations occurred in the actual execution.
- Note:
 - Linearizability concerns only the **interleaving of individual operations** and is not intended to be transactional
 - Linearizable execution **may break** application specific notions of **consistency** if concurrency control is not applied

Example

- Two servers **A** and **B** have replicated data of accounts **X** and **Y**
- Suppose that both accounts have balance '**0**' initially

Client 1

setBalance_B(x,1)

Client 2

getBalance_A(y) → 0

getBalance_A(x) → 0

setBalance_A(y,2)

There is a delay in getting updates from server B to server A

Is it linearizable?

Example

- Two servers **A** and **B** have replicated data of accounts **X** and **Y**
- Suppose that both accounts have balance '**0**' initially

Client 1

Client 2

setBalance_B(x,1)

getBalance_A(y) $\longrightarrow$ 0

getBalance_A(x) $\longrightarrow$ 0

setBalance_A(y,2)

There is a delay in getting updates from server B to server A. The interleaving of operations is
setBalance_B(x,1) getBalance_A(y) $\longrightarrow$ 0, getBalance_A(x) $\longrightarrow$ 0, setBalance_A(y,2)

- Not linearizable, setBalance_B(x,1) is before getBalance_A(x) $\longrightarrow$ 0 , inconsistency in results
- Linearizable if we had getBalance_A(x) $\longrightarrow$ 1

Linearizability

Linearizability

In practical systems, each operation should appear to take effect instantaneously at some moment between its start and completion.

Operations complete within a given time (shaded area). For a write operation, the timestamp value can be anywhere in the shaded region.



Operations with overlapping shaded regions can appear in any order, but non-overlapping operations should maintain their ordering: e.g., W(x)a precedes W(x)b, but W(x)a and W(y)b can appear in any order

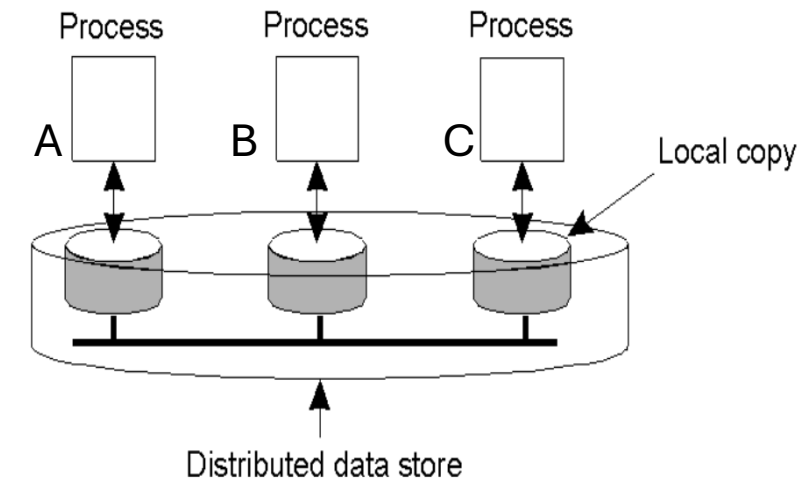
Possible ordering of operations	Result	
$W_1(x)a; W_2(y)b; W_1(y)a; W_2(x)b$	$R_1(x)b$	$R_2(y)a$
$W_1(x)a; W_2(y)b; W_2(x)b; W_1(y)a$	$R_1(x)b$	$R_2(y)a$
$W_2(y)b; W_1(x)a; W_1(y)a; W_2(x)b$	$R_1(x)b$	$R_2(y)a$
$W_2(y)b; W_1(x)a; W_2(x)b; W_1(y)a$	$R_1(x)b$	$R_2(y)a$

W_i(x)a: write the value a to x at processor P_i

R_i(x)b: read the value b of x at processor P_i

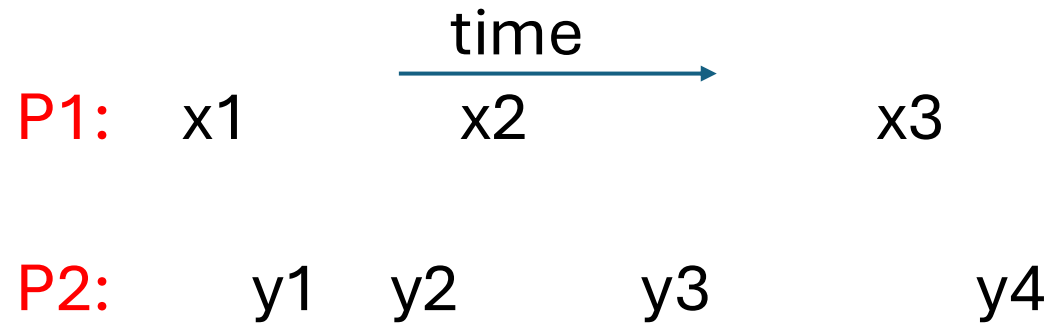
Sequential Consistency

- **Definition:** if for any transaction, there is some interleaving of the series of operations issued by all the clients that satisfies the following two criteria:
 - The interleaved sequence of operations meets the **specification** of a (single) correct copy of the objects.
 - The order of operations in the interleaving is consistent with the **program order** in which each individual process executed them. That is, for each process, the interleaving of operations has the same order for the process' operations as process description.
- All processes should see identical ordering of operations by maintaining real-time ordering for operations of each individual process, but they may not maintain real-time ordering across processes.
- Difference from strict consistency
 - no reference to the most recent time
 - absolute global time does not play a role



Example for Sequential Consistency

- Two processes P1 and P2 execute different operations sequentially



Virtual interleaving of operations: P1 and P2 operations are in the same order as their execution

x1 y1 x2 y2 y3 x3 y4 (x2, y2 do not follow real-time order)

This ordering is valid when the effects of y2 are seen later than the effects of x2. There is no strict consistency.

Example

- Two servers **A** and **B** have replicated data of accounts **X** and **Y**
- Suppose that both accounts have balance '**0**' initially

Client 1

Client 2

setBalance_B(x,1)

getBalance_A(y) $\longrightarrow$ 0

getBalance_A(x) $\longrightarrow$ 0

setBalance_A(y,2)

There is a delay in getting updates from server B to server A.

- **Is it Sequentially consistent?**

Example

- Two servers **A** and **B** have replicated data of accounts **X** and **Y**
- Suppose that both accounts have balance '**0**' initially

Client 1

Client 2

setBalance_B(x,1)

getBalance_A(y) → 0

getBalance_A(x) → 0

setBalance_A(y,2)

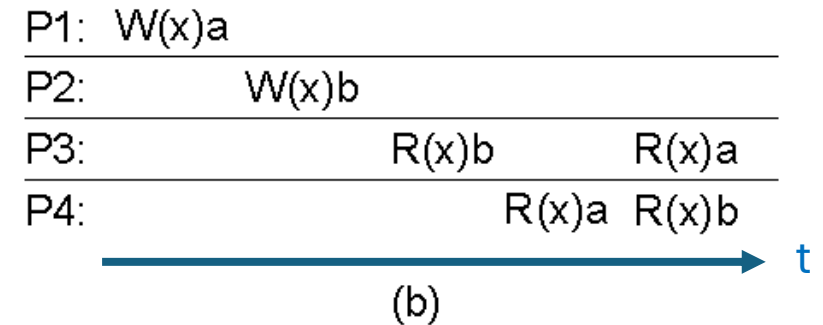
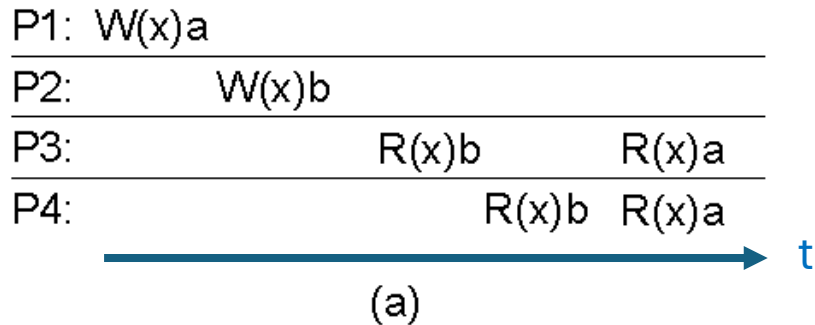
There is a delay in getting updates from server B to server A

- Sequential consistency is there: we can get the same outcomes by using the following interleaving of operations in which ordering of operations is preserved for each client

getBalance_A(y) → 0, getBalance_A(x) → 0,

setBalance_B(x,1) , setBalance_A(y,2)

Example of Sequential Consistency



- a) Fig a, a sequentially consistent data store. (P3 and P4 **see** the same order)
- b) Fig b, a data store that is not sequentially consistent. (P3 and P4 **don't see** the same order of events)
- c) Note: It doesn't matter when the events actually took place
- d) It does matter if all processes see them in the same order

Note: Operations that are linearizable also fulfill sequential consistency.

- We can implement sequential consistency when we do not have a global clock or clock synchronization; processes can exchange messages to decide a single ordering for operations as per the sequential consistency.

Causal Consistency

- The causal consistency model represents a weakening of sequential consistency in that it makes a distinction between events that are potentially causally related and those that are not.
- Necessary condition:
 - **Writes** that are ***potentially*** causally related must be seen by all processes in the same order. Concurrent writes may be seen in a different order on different machines. (Hutto and Ahamad, 1990)

- If event B is caused or influenced by an earlier event A, causality requires that everyone first see A and then B
- *Concurrent*: operations that are not causally related

Example of Causal Consistency

P1:	W(x)a		W(x)c	
P2:		R(x)a	W(x)b	
P3:		R(x)a		R(x)c R(x)b
P4:		R(x)a		R(x)b R(x)c



- This sequence is allowed with a causally-consistent store, but not with sequentially or strictly consistent store.
- W(x)a and W(x)b are causally related as there is R(x)a before W(x)b at P2; the effect of W(x)a (applied at P1) is seen at P2 before applying W(x)b
- W(x)b and W(x)c are concurrent as there is no intervening read operation
 - So, all processes don't see them in the same order
- P3 and P4 read the values 'a' and 'b' in order as they are potentially causally related. No 'causality' for the value 'c'
- This is not sequentially consistent though
 - As P3 and P4 see the values in different order

Eventual Consistency

- In many systems, one or a few processes perform updates
 - How frequently should these updates be made available to other read-only processes?

Examples:

- DNS: single naming authority per domain
- In many stems, only sys-admins update the database, users only read data. This is true for webpages; inconsistent files may not have significant consequences.
- Cloud storage: Dropbox, OneDrive, iCloud all use eventual consistency

Eventual consistency: in the absence of updates, all replicas converge towards identical copies

- Only requirement: an update should eventually propagate to all replicas
 - Note that we weaken consistency properties to improve efficiency/performance of the system: strict consistency results in lower performance as it enforces stricter conditions

Reference

- **Chapter 7 of**
Distributed Systems, 4th Edition,
Maarten van Steen and Andrew
Tannenbaum.
- **Chapters 17 and 18 of**
Distributed systems : concepts and
design / George Coulouris ... [et al.],
Addison-Wesley (Pearson), c2012.



COMP2211

Distributed Systems

Topic7: Fault Tolerance

Thiru Vasantam

This Topic

Aim

Fault Tolerance

- Non-functional (QoS) requirement; operations should be transparent and preferably autonomous

Experience from KFC

Solutions:

- Approaches Based on Redundancy
- Error Recovery

Measurement of Availability

Example: KFC Crisis

- 14 Feb 2018:
 - Broke up with Bidvest
 - Engaged with DHL
- 18 Feb 2018: ~~900~~ → 266
- Violate distributed system rules:
 - Single point of failure:
One centralized warehouse at Rugby
 - No replication: No matter active/passive
 - An accident near the warehouse caused network congestion
- End up with: **system crash**
- https://ceres.shop/blog/the-saga-of-the-2018-kfc-chicken-shortage/?srsltid=AfmBOop4_q5YYNgdV61Hq3AXP57UeX80qLJLsLfPDPtxWs0l6wBbNTz-





Fault Tolerance and Redundancy

Nature of Distributed Systems (DS)

- DS comprises independent components
- System behavior may be less predictable and prone to failures
- If 'p' is the probability for each component to fail then $1 - (1 - p)^n$ is the probability for a DS to have a failed component when it has 'n' components, ex: n=5, p=0.1, $1 - (1 - p)^n = 0.409$
- Importance to mission-critical applications: e.g., medical, aviation, banking, etc.
 - Failures are very costly
- **Fault Tolerance**
 - Ability of system to continue **error-free operation** even in the presence of unexpected fault
 - i.e., increase system availability



Measurement of System Quality

Reliability (time duration without failure)

- the extent to which a system yields expected results on repeated trials
- measured by **mean-time-between-failures** (MTBF)

Availability (percentage of time being ready to use)

- fraction of time the system yields expected results
- reduced by downtime, repair, & preventive maintenance

$$A = \frac{MTBF}{MTBF + MTTR}$$

- where MTTR is **mean-time-to-repair**

Measuring Availability (I)

"Myth of the 9s" ...

As we move to higher levels of availability, costs increase very rapidly!

% uptime	Downtime per year	Downtime per week
98%	7.3 days	3h 22min
99%	3.6 days	1h 41min
99.8%	17h 30min	20min 10sec
99.9%	8h 45min	10min 5sec
99.99%	52.5min	1min
99.999%	5.25min	6sec
99.9999%	31.5sec	0.6sec

Measuring Availability (III)

All downtime is not created equal:

- Single 30-min outage at ebay.co.uk:
- 99.994% availability ... apparently ☹
- Friday, evening hours, in early December vs Sunday, early morning hours, in July ??
- 30 1-min outages equal 1 30-min outage ??

What about hours during which 100% uptime may be required?

Max. outages are more important than the average uptime.

Terminology

Term	Description	Example
Failure	A component is not living up to its specifications	Crashed program
Error	Part of a component that can lead to a failure	Programming bug
Fault	Cause of an error	Sloppy programmer

Types of Failures

- **Omission failures:** a component fails to take an action that it should have taken
- **Commission failure:** a component takes an action that it should not have taken

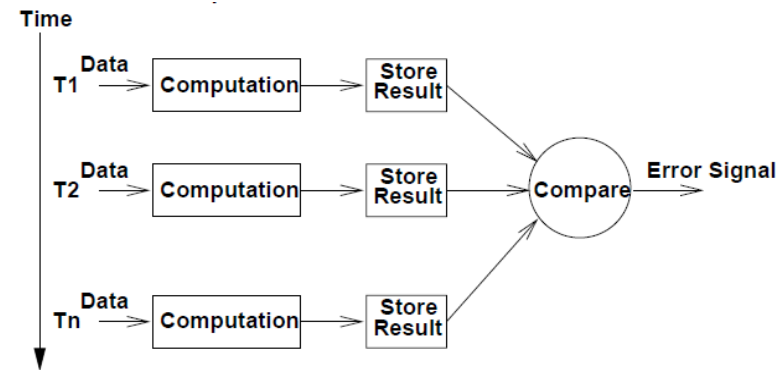
Type	Description of server's behavior
Crash failure	Halts, but is working correctly until it halts
Omission failure <i>Receive omission</i> <i>Send omission</i>	Fails to respond to incoming requests Fails to receive incoming messages Fails to send messages
Timing failure	Response lies outside a specified time interval
Response failure <i>Value failure</i> <i>State-transition failure</i>	Response is incorrect The value of the response is wrong Deviates from the correct flow of control
Arbitrary failure	May produce arbitrary responses at arbitrary times

Approaches based on Redundancy

- Apply duplication to increase system reliability
- System Architecture approach
 - Incorporate Active or Passive replication
 - Design server configuration and number of replicated servers
 - Could be expensive due to requiring extra hardware
- Operational approach
 - Replicate system operations to offer fault tolerance
 - Time Redundancy
 - Component Redundancy
 - Information Redundancy
 - Communication Redundancy

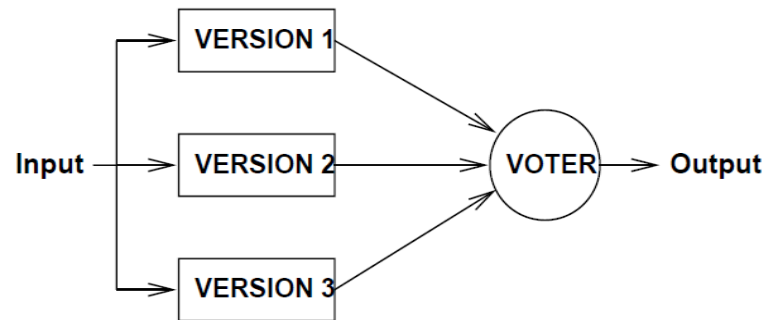
Time Redundancy

- **Time redundancy:** Design a system such that an action can be performed again if anything went wrong. Typically used when faults are transient (appear once and disappear) or intermittent (repeatedly appear and disappear). Examples are
 - Transient errors are: brief network congestion, packet loss;
 - Intermittent faults are: loose connections, some software bugs, condition dependent errors etc
- Perform the same operation multiple times
 - No fault if getting the same result each time
 - Detect temporary faults but not permanent ones
 - Impact system performance



Component Redundancy

- Replicate component and compare outputs
 - Introduce two or more independent running components which provide the same functionalities
 - Impose little or no performance impact
- N-Version Programming (NVP)
 - **Design diversity:** implementing multiple versions of the program
 - Tolerate hardware and software faults, but not correlated faults (like storing in input data in cache and then there could be errors when retrieving it)



Information Redundancy

- **Information redundancy**: Add extra bits to data units so that errors can be recovered when bits are garbled.
- Encode outputs with error detecting or correcting code
 - Code selected to **minimize the redundancy** for a class of faults
 - e.g. **Parity check** – breaks the data into "words" with a fixed number n of bits, and then computes the 'exclusive or' of all those words
 - e.g. **Checksum** – a fixed-size datum computed from an arbitrary block of data for detecting accidental errors that may have been introduced during its transmission or storage
- Advantage
 - Less hardware is required than replicating module
 - Support fault detection
- Drawback
 - Added complexity in design
 - fault recovery capability may be limited

Communication Failures

Client is unable to locate server	Client request to server is lost
• Use an exception handler (programming language dependent) • Check out available / updated servers from a directory service	• Apply timeout to await server reply, then re-send (*Note: Applicable to idempotent operation: has no additional effect if executing twice or more) • If multiple requests appear to get lost assume 'cannot locate server' error
Server crashes after receiving client request	Server reply to client is lost
• Server may stop before or after returning the info. or before ACK • Store user request (FE / MO Middleware) • Rebuild or use alternative Server, retry request • Give up and report failure (for 'at most once' request)	• Apply timeout to await server reply (same as above)



From Fault to Recovery

General Workflow towards Fault Tolerance

- Error (or fault) detection & diagnosis
- Failure Isolation
 - When a failure occurs, the system must be able to isolate the failure to the offending component
 - Remove faulty components (→ system reconfiguration)
- Error Containment
 - Confine the effects of hardware or software faults to the region where they occur
 - Prevent further damage & error propagation
- Recovery
 - Move the system to a state that does not contain the error

Types of Error Recovery

- Backward recovery
 - Move the system back to a failure-free state
 - Focus: Make sure a system is correct before continuing to operate
- Forward recovery
 - Find a new state from which the system can continue operation
 - Focus: Avoid a system to stop even if failure occurs

Implement Backward Recovery

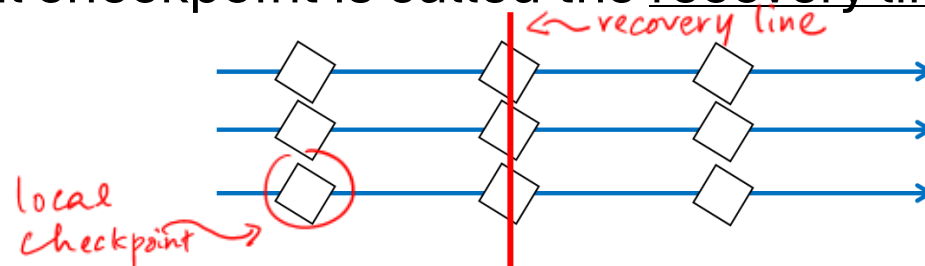


- **Checkpointing:**

- Each DS component periodically saves its state, which contains sufficient information to restart component execution

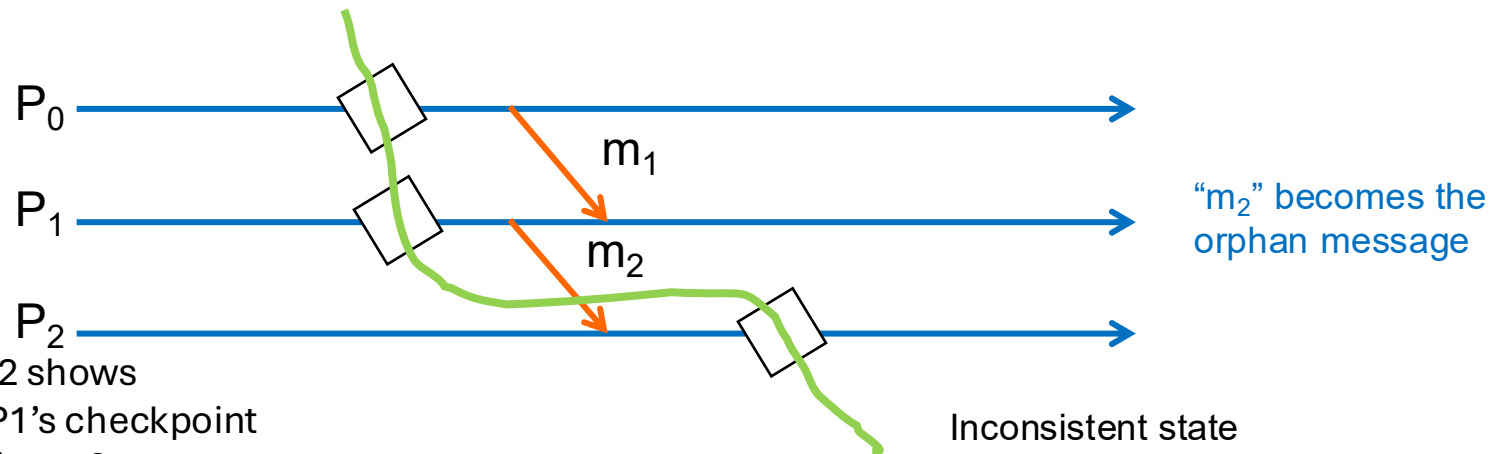
- **Global checkpoint:**

- To support state saving, a consistent global checkpoint can be set up, which comprises a set of N local checkpoints, one from each DS component, forming a consistent system state
- Component execution can be restarted upon a failure by following the saved checkpoints
- The most recent consistent global checkpoint is called the recovery line



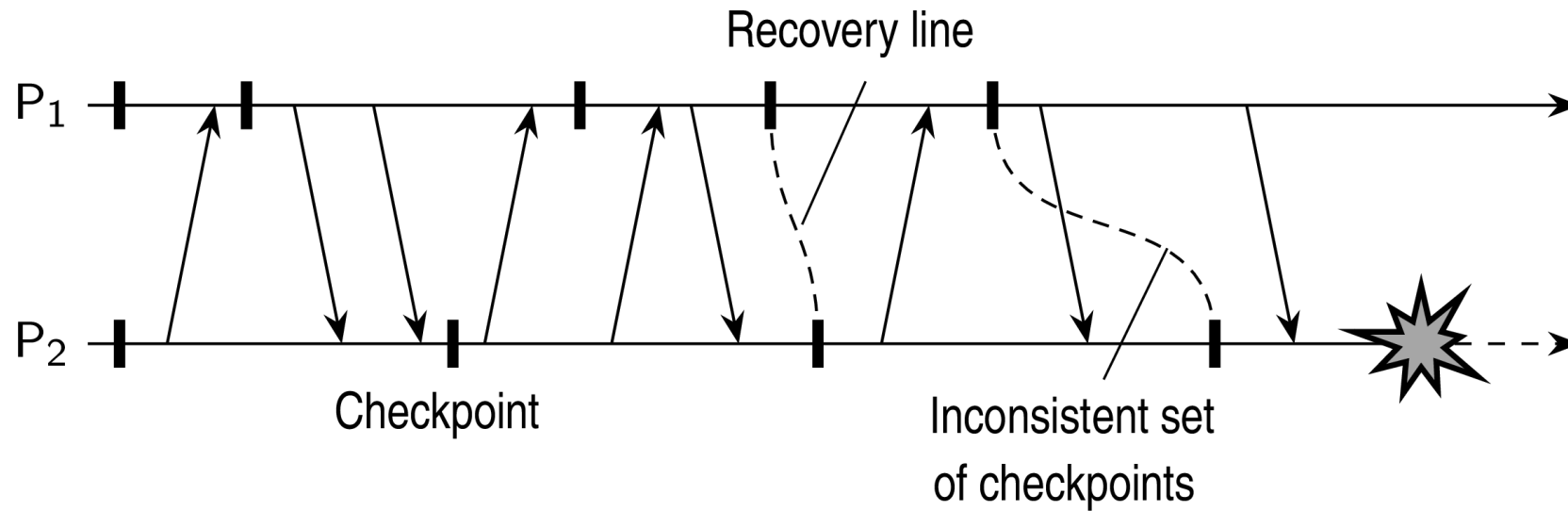
How to set up checkpoints?

- Consistent system state:
 - If the state of a process reflects a message receipt, then the state of the corresponding sender reflects sending that message. It is acceptable if only the sender records a message, but not the receiver.



- The checkpoint of P_2 shows m_2 is received, but P_1 's checkpoint does not show sending m_2 .
e.g., P_1 can repeat the same computation and send updates to P_2 again, such as $x=x+1$ for some data x .
- Goal of Backward Recovery
 - Bring the system into a consistent state when inconsistencies occur because of a failure

Example



- The second set of checkpoints (i.e., the second dashed line) form the recovery line as it has no orphan messages

Types of Checkpointing

- **Uncoordinated checkpointing**

- Each process takes its checkpoints independently
- Automatic; convenient; possible useless checkpoints; domino effect

- **Coordinated checkpointing**

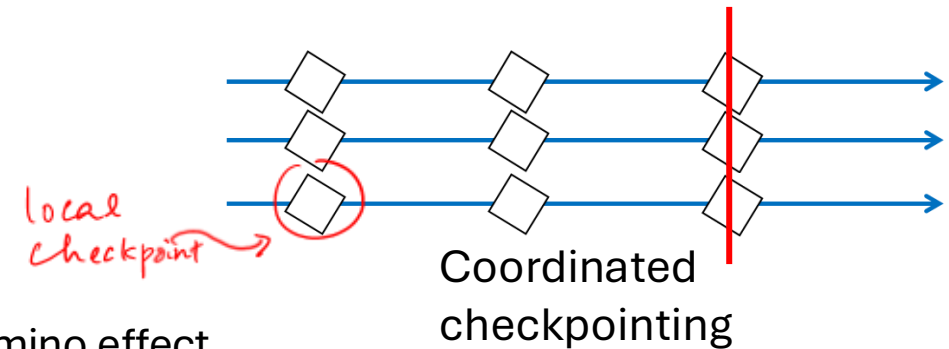
- Processes take coordinated checkpoints in order to save a system-wide (global) consistent state by taking checkpoints at the same time across processes
- Less storage overhead; avoid domino effect; performance issue as all the processes stop their operations to perform checkpointing

- **Communication-induced checkpointing**

- Force each process to take checkpoints based on information piggybacked on the application messages it receives from other processes (add forced checkpoints → recovery line)
- Avoid domino effect; allow local checkpoints

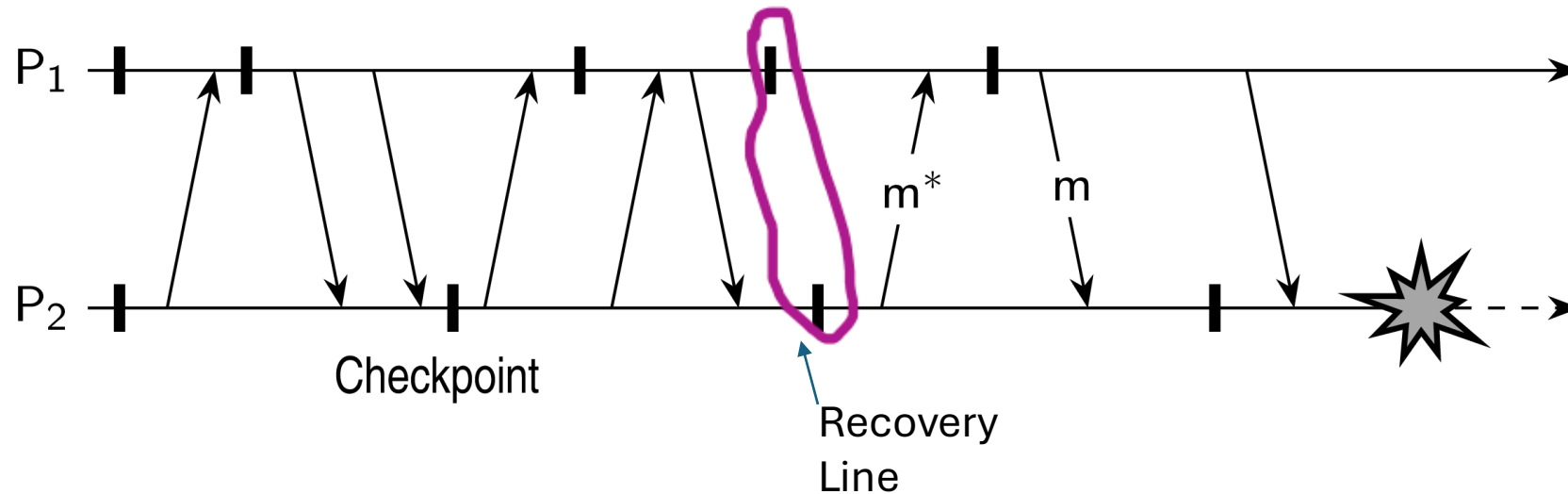
- **Message-Logging**

- Take infrequent checkpoints
- Log all messages between checkpoints to local stable storage
- To recover, simply replay messages from previous checkpoint to avoid repeating operations/computations



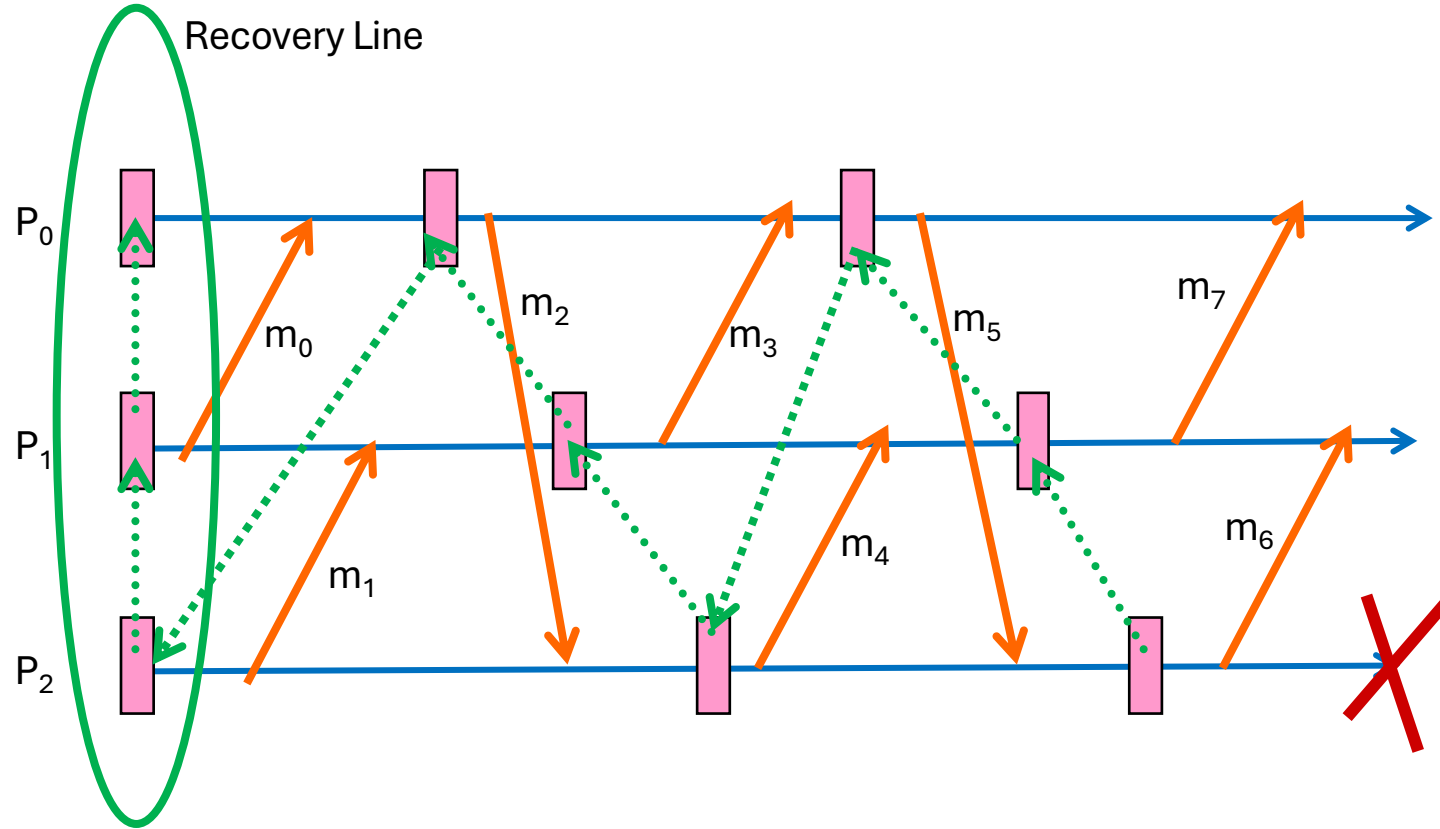
Issue of Backward recovery

- Restarting the system from a recovery line may force the system to repeat some operations/computations



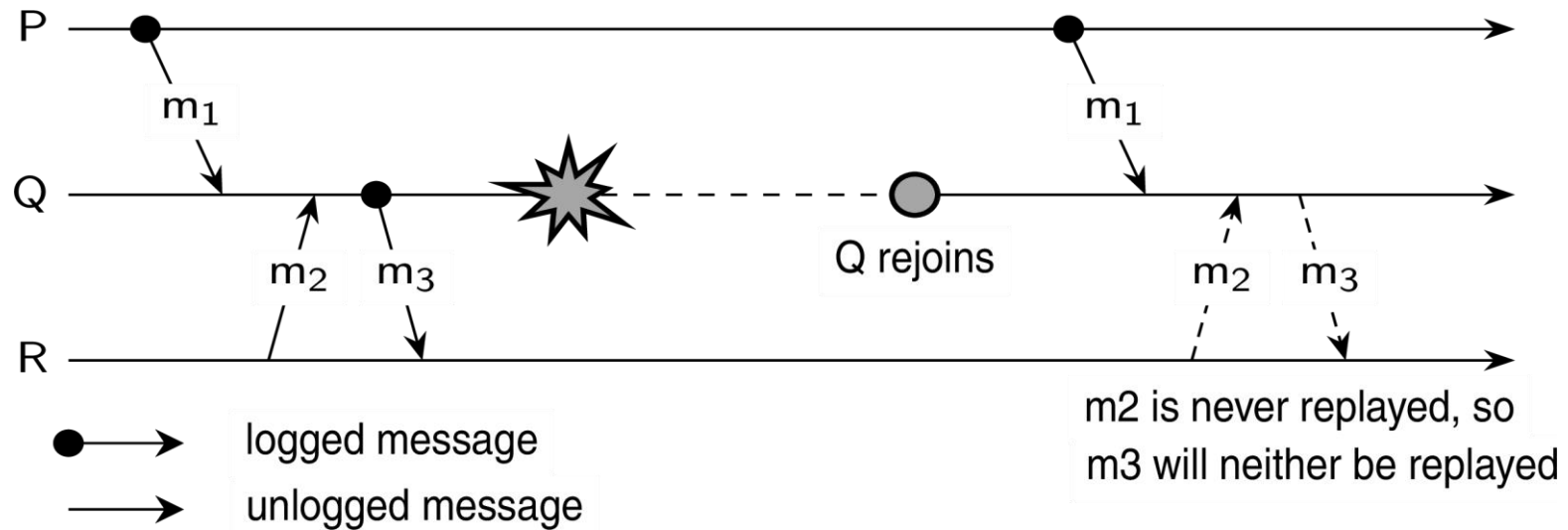
Main Issue of Backward Recovery

- Sometimes the recovery line may rollback to the initial point



Domino Effect: Cascaded rollback which causes the system to rollback to too far in the computation (even to the beginning), in spite of all the checkpoints

Message Logging



m_1 message is replayed, no need to execute this task again.

Implement Forward recovery

- **Methods to avoid stopping the system:**
 - **Self-checking components**
 - Switch from a failed to a non-failed component executing the same task-code
 - **Fault masking**
 - Error compensation is continuously applied, e.g., Voting schemes (address Byzantine fault)
 - **Error compensation / resilience**
 - Based on algorithm that uses redundancy/diversity
 - ... multiple processes, executing in parallel, sending redundant or rearranged data, that deliver potential results
 - e.g., Packet interleaving (rearranging data packets before transmission to reduce the impact of bursty packet loss), piggybacking (attaching acknowledgement to the next packet)
 - **Data prediction**
 - Simulate application response (e.g., online gaming where we can estimate positions and movements when we lose some data)

Discussion: Comparison

- Backward recovery
 - Requires no knowledge about the error
 - We need to maintain only some prior error-free state
 - Application-independent
 - Limitations: resource / time consumption, domino effect
- Forward recovery
 - Efficient in terms of time and storage space
 - Requires the knowledge of the error to take appropriate actions
 - Application-dependent
 - Used when significant system delay is not acceptable

Reference

- **Chapter 8 of**
Distributed Systems, 4th Edition, Maarten van Steen and Andrew Tannenbaum.
- A. Avizienis, J.C. Laprie, B. Randell, C. Landwehr:
‘Basic Concepts and Taxonomy of Dependable and Secure Computing’,
IEEE Trans. on Dependable and Secure Computing, vol. 1, no. 1, pp. 11-33, Jan-March 2004.

